

Dengue Exercise (Part 2): From Least Squares to ARGO

SISMID 2026 • Day 2, 1:30 • From the Dengue Exercise to ARGO

In the previous exercise you fit a *static* model to predict dengue cases in Mexico using training data from 2004–2006: a single, unchanging line you use to predict all future cases. In this exercise you fit a *dynamic* model that is retrained at each prediction step, then add more features until the model becomes a basic **ARGO** model (AutoRegression with Google search data). A short final part sketches the network (**ARGONet**) idea.

You may work in **Python** or **R**, and either drive a coding agent (Codex, Claude Code, or Antigravity CLI) with a prompt per part, or fill in the pre-filled notebook. Data: `MX_Dengue_trends.csv` (Dengue CDC = reported cases; `dengue`, `sintomas de dengue`, `mosquito`, `dengue sintomas` = Google search interest), from <https://github.com/MIGHTE-lab/SISMID25>.

- (a) **Dynamic (sliding-window) training.** In a sliding-window approach, the original training period (Jan. 2004 – Dec. 2006) fits a least squares model and predicts the next time step (Jan. 2007). For the next prediction, the training period shifts by one month (Feb. 2004 – Jan. 2007) and the model is retrained. Continue this for each prediction from 2007–2011 and store the predictions in a vector.
- (b) **Compare to static.** Plot the cases predicted by the dynamic method against the predictions from the static model and the ground truth.
- (c) **Score both.** Compute the mean square error (MSE, or RMSE) of both methods against the ground truth. Which is better, and why?
- (d) **Add covariates.** Instead of just `dengue`, add the remaining Google search terms as predictors in the dynamic model. How does the error change? With many correlated terms, try **Lasso** (ℓ_1 -penalized regression) so useless terms drop out on their own.
- (e) **Add autoregression (this is ARGO).** Use the fact that the model is refit each step to add past observations of the ground truth as features. For the Jan. 2007 prediction, add the observed case counts from Nov. 2006 and Dec. 2006 (a Google + AR2 model). Implement this dynamic model with autoregression and compare its error to the others. This combination,

$$y_t = \mu + \sum_{k=1}^2 \beta_k y_{t-k} + \sum_j \alpha_j x_{j,t} + \varepsilon_t,$$

trained dynamically, is a basic **ARGO** model.

Hint: shift the case-count column and stack it onto the feature matrix; the first one or two windows may have fewer than 36 observations.

- (f) **Discuss and extend (ARGONet).** You should see the error fall at each step: static $\rightarrow$ dynamic $\rightarrow$ +terms $\rightarrow$ +autoregression. Now suppose you had *several* locations (states or counties) instead of one country. **ARGONet** adds a network step: borrow signal from connected neighbors, for example by adding a neighbor’s recent activity (or its own ARGO estimate) as an extra predictor, choosing neighbors by shared movement or short-lag correlation. Sketch, in words or code, how you would add one neighbor’s lagged signal to the model above, and say when you would expect it to help.

Stretch: refit ARGO with an ℓ_1 penalty and a two-year rolling window (the full ARGO of Yang, Santillana & Kou, PNAS 2015), and compare against the plain least-squares version.